v1.0.0
20260401

Programmazione Concorrente e Distribuita

Ingegneria e Scienze Informatiche - UNIBO
a.a 2025/2026
Prof. Alessandro Ricci

[Lab Notes]

REACTIVE PROGRAMMING IN JAVA: RXJAVA

RXJAVA

an Observable is a provider that emits a stream of data or events over time.

- RxJava is a Java VM implementation of Reactive Extensions
  - https://github.com/ReactiveX/RxJava
- It extends the *observer pattern* to support sequences of data/events and adds operators that allow you to compose sequences together declaratively
  - abstracting away concerns about things like low-level threading, synchronization, thread-safety, concurrent data structures, and non-blocking I/O
- Last version
  - RxJava 1.x
    - implementation of Rx on top of Java
  - RxJava 2.x (2017)
    - compatible with *ReactiveStreams* initiative
  - **RxJava 4.x (2026)**
    - last major release

RXJAVA AND REACTIVE STREAM

- Reactive Streams
  - *"The purpose of Reactive Streams is to provide a standard for asynchronous stream processing with non-blocking backpressure"*
- Reactive Streams is a standard and specification for Stream-oriented libraries for the JVM that
  - process a potentially unbounded number of elements in sequence,

    A differenza di una List che ha una dimensione finita, un Reactive Stream è pensato per essere infinito. Il sistema non deve mai provare a caricare "tutto" in memoria, perché "tutto" non esiste o è troppo grande. I dati devono fluire come in un tubo, non come in un secchio.
  - asynchronously passing elements between components,
  - with mandatory non-blocking *backpressure*

    Lo standard stabilisce che è il Subscriber a dire al Publisher quanti dati è pronto a ricevere. Non è un modello "Push" selvaggio, ma un "Push controllato". Il Subscriber invia un segnale: "Ehi, mandami i prossimi 10 elementi". Il Publisher invia solo quei 10 e poi si ferma finché non riceve una nuova richiesta. "Non-blocking" significa che questa negoziazione non ferma i thread; avviene tramite segnali asincroni leggeri.
- Link
  - https://github.com/reactive-streams/reactive-streams-jvm/

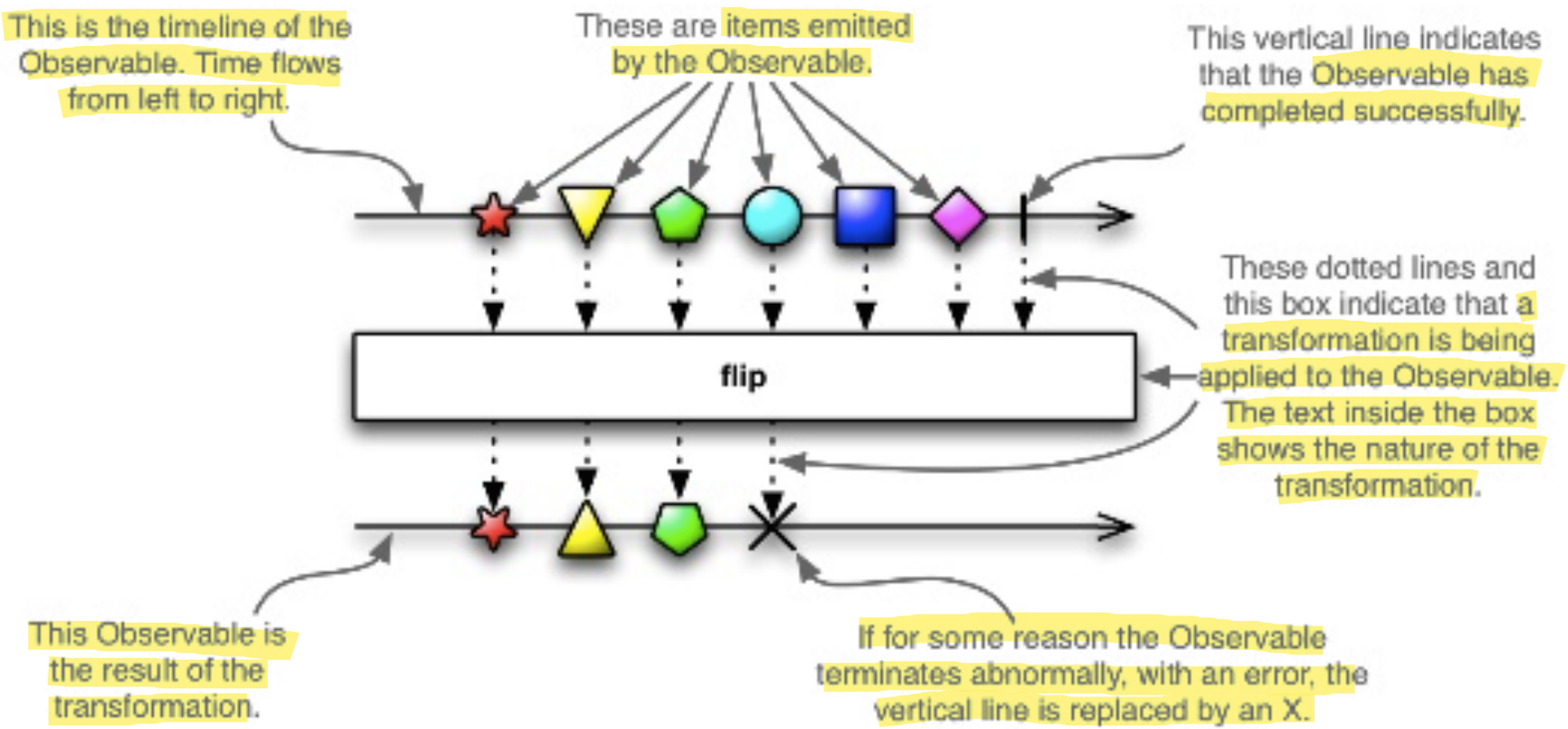
In un sistema reattivo, hai diversi pezzi (componenti) collegati tra loro. Se il "Componente A" invia un dato al " Componente B", lo fa in modo asincrono. Questo permette ad A di non restare fermo ad aspettare che B finisca di elaborare. A "spara" il dato e torna a fare il suo lavoro.

RX-* OPERATORS

Gli operatori sono funzioni che permettono di manipolare i flussi di dati senza mai dover scrivere cicli for o gestire stati manuali.

- Detailed documentation at http://reactivex.io/documentation/operators.html
- **creating** observables Servono a far nascere uno stream dal nulla o da dati esistenti.
  - operators that originate new observables.
- **transforming** observables Modificano gli elementi mentre passano nel "tubo".
  - operators that transform items that are emitted by an observable
- **filtering** observables Decidono cosa far passare e cosa scartare.
  - operators that selectively emit items from a source observable.
- **combining** observables Uniscono più flussi in uno solo.
  - operators that work with multiple source observables to create a single observable
- **conditional** and boolean operators
  - operators that evaluate one or more observables or items emitted by observables
- **mathematical** and aggregate operators Operano sull'intero flusso per ridurlo a un valore solo (spesso trasformando lo stream in un valore singolo quando si completa).
  - operators that operate on the entire sequence of items emitted by an observable
- **backpressure** operators Questi sono operatori "di sicurezza" per quando la sorgente è troppo veloce (come visto nello standard Reactive Streams).

OPERATORS: MARBLE DIAGRAMS



TRASFORMING OPERATORS

(non ci sono modifiche di stato, non ci sono side-effect: dato un flusso asincrono, ne creo un altro; esempio: Map)

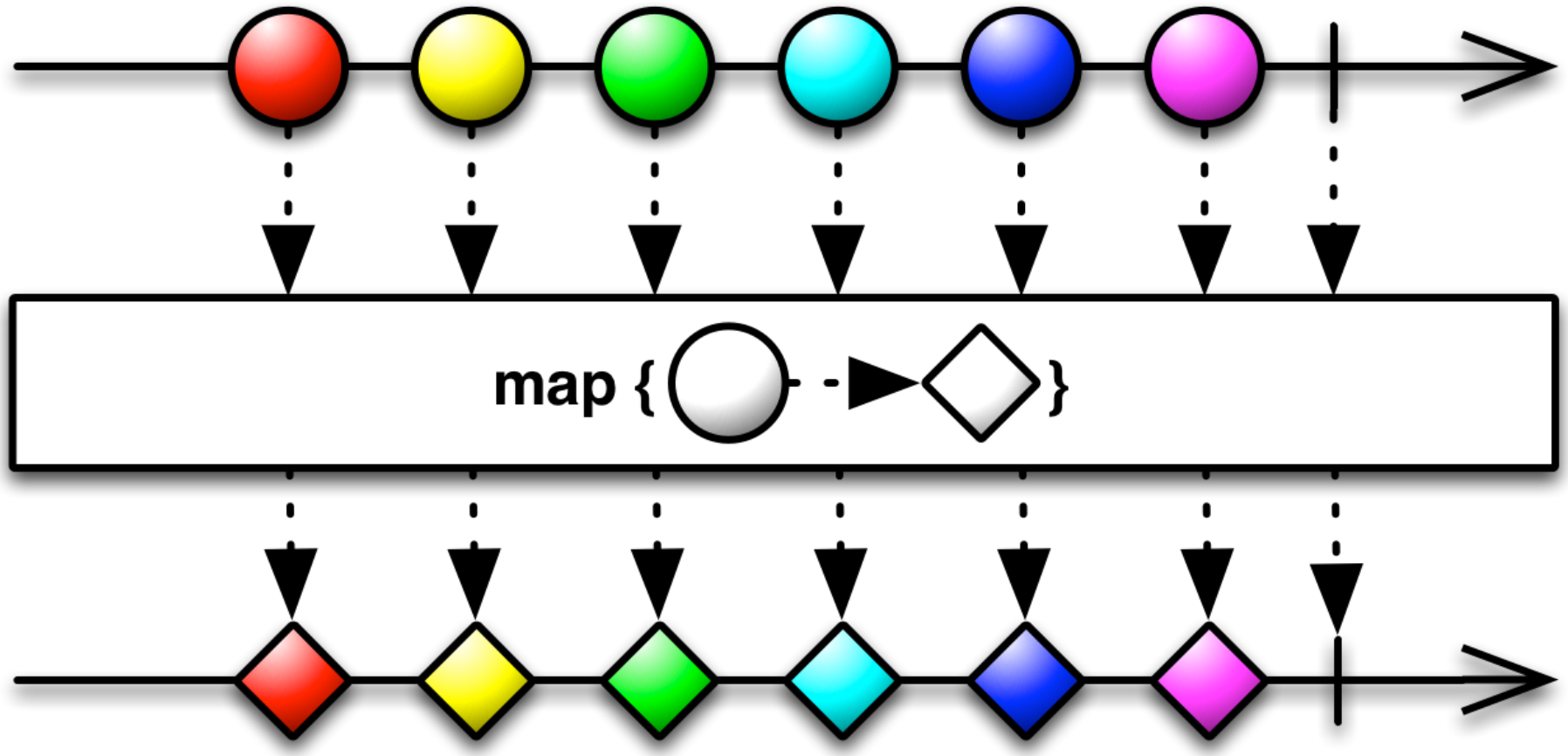
- **Map** — transform the items emitted by an Observable by applying a function to each item

  Prende un valore dal flusso asincrono e lo trasforma in un altro valore in un altro flusso asincrono.
- **Scan** — apply a function to each item emitted by an Observable, sequentially, and emit each successive value

  Ogni elemento del flusso asincrono può generare un flusso asincrono, a sua volta, di elementi e poi voglio che tutti quegli elementi vengono messi insieme in un unico flusso asincrono
- **FlatMap** — transform the items emitted by an Observable into Observables, then flatten the emissions from those into a single Observable
- **Buffer** — periodically gather items from an Observable into bundles and emit these bundles rather than emitting the items one at a time
- **GroupBy** — divide an Observable into a set of Observables that each emit a different group of items from the original Observable, organized by key
- **Window** — periodically subdivide items from an Observable into Observable windows and emit these windows rather than emitting the items one at a time

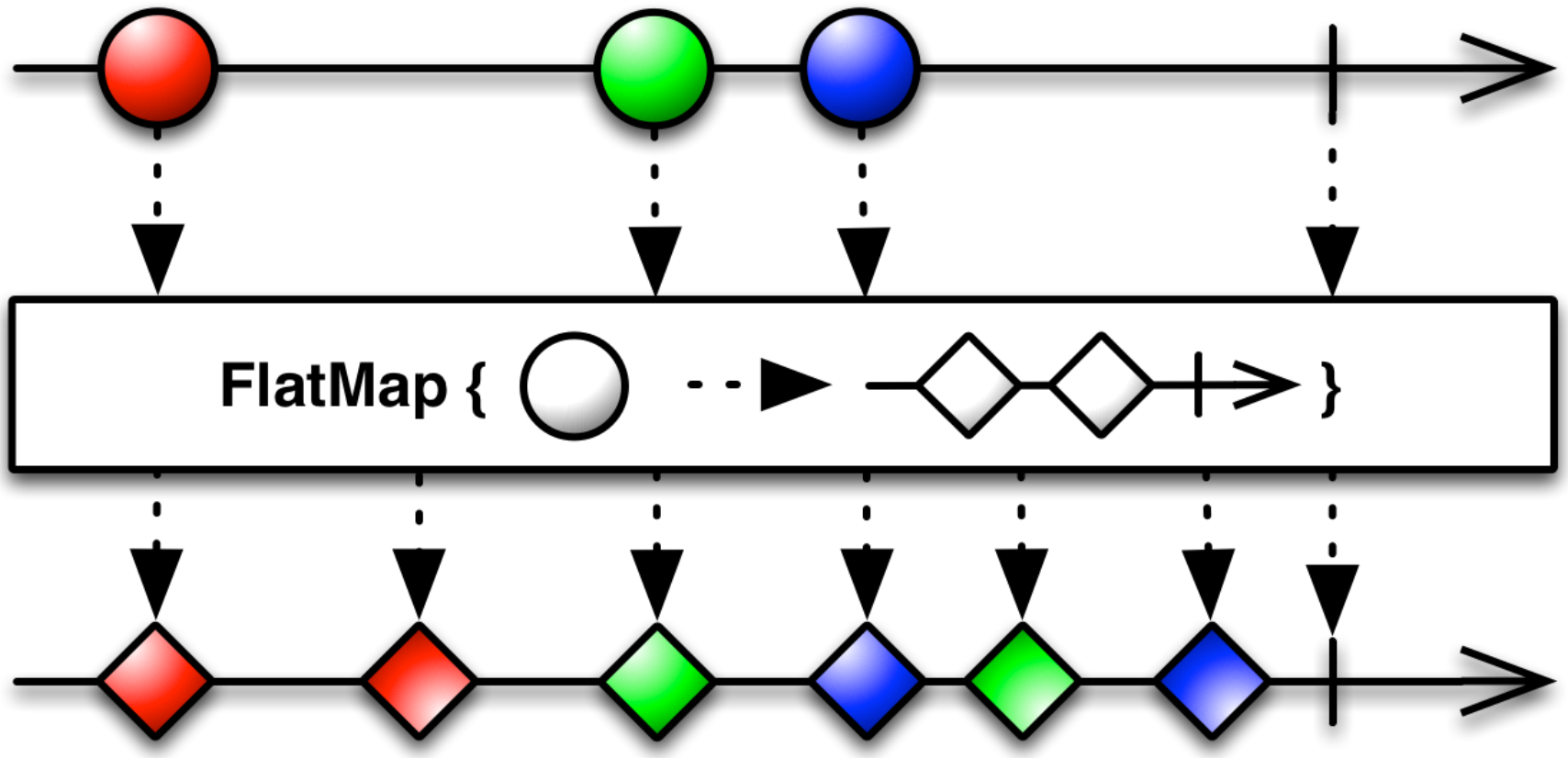
Tutti i flussi asincroni sono concettualmente continuativi e senza modifiche. Ho solo un'espansione della conoscenza dei dati; non ho una modifica. Questi sono approcci che, se governati bene, creano sistemi molto efficienti perchè non devono mai gestire modifiche e quindi corse critiche

OPERATORS: MAP



FLAT MAP

A prescindere dal colore, vengono generati due rombi per ogni palla
(ogni palla genera un flusso) e poi vengono messi nello stesso flusso



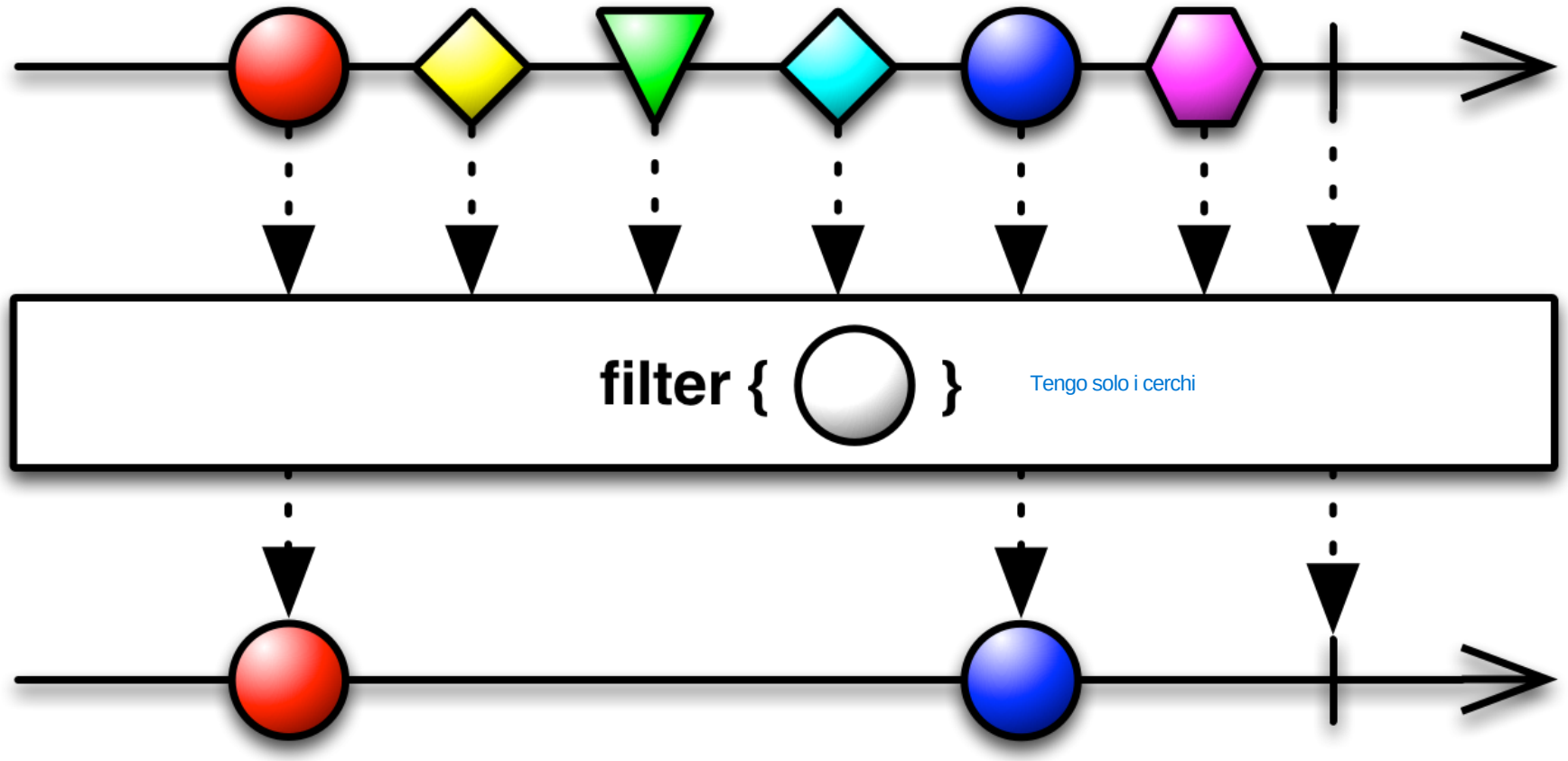
FILTERING OPERATORS

Ripensare tutte le computazioni in termini di flussi, grazie agli operatori di filtering

- **Filter** — emit only those items from an Observable that pass a predicate test
- **Skip** — suppress the first n items emitted by an Observable
- **Take** — emit only the first n items emitted by an Observable
- **Debounce** — only emit an item from an Observable if a particular timespan has passed without it emitting another item
- **Distinct** — suppress duplicate items emitted by an Observable
- **ElementAt** — emit only item n emitted by an Observable
- **First** — emit only the first item, or the first item that meets a condition, from an Observable
- **IgnoreElements** — do not emit any items from an Observable but mirror its termination notification
- **Last** — emit only the last item emitted by an Observable
- **Sample** — emit the most recent item emitted by an Observable within periodic time intervals
- **SkipLast** — suppress the last n items emitted by an Observable
- **TakeLast** — emit only the last n items emitted by an Observable

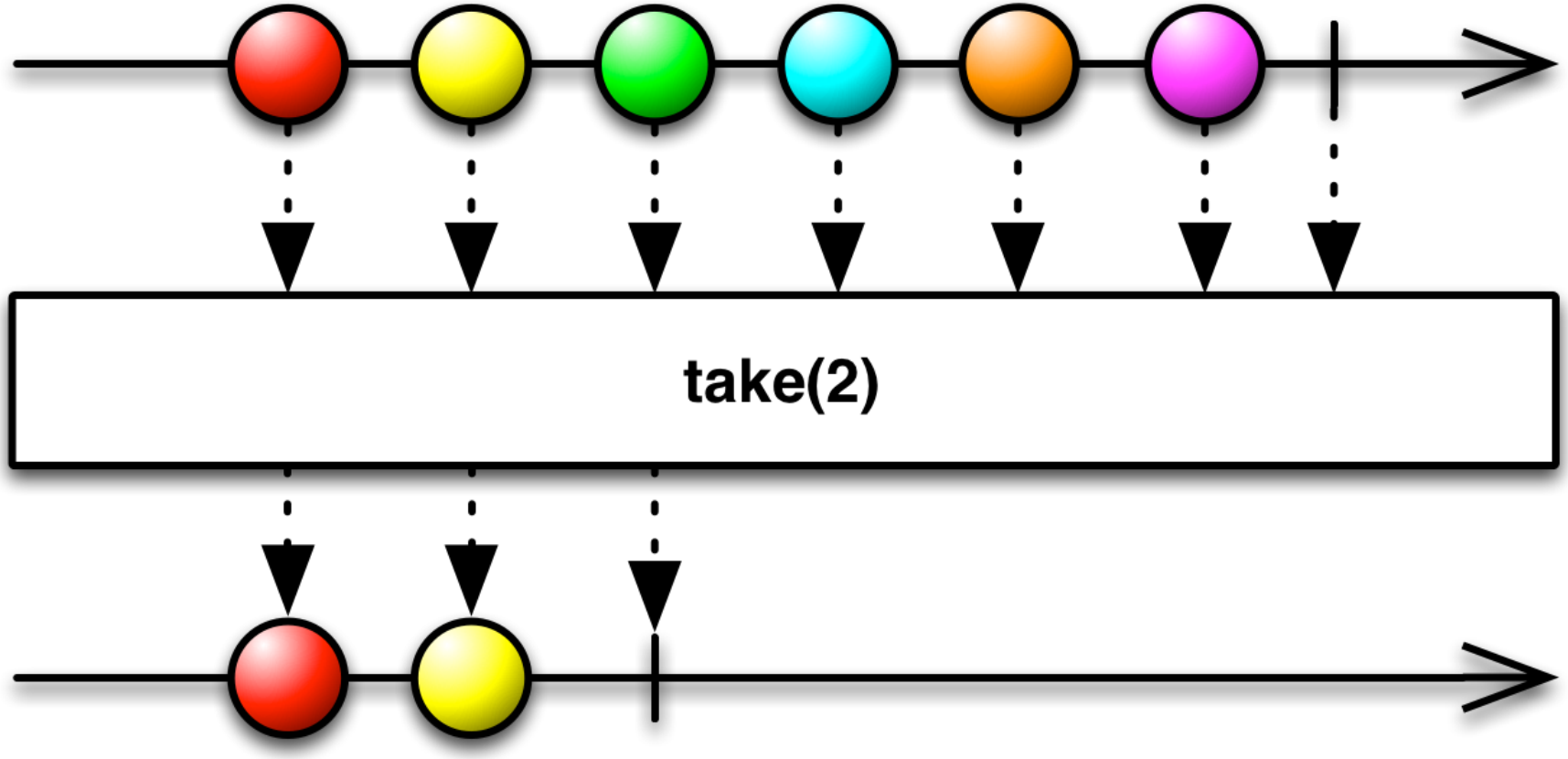
FILTER

Dato un flusso asincrono di elementi, voglio selezionare solo quelli che rispettano la condizione del filter



In queste slide, stiamo lavorando con flussi di dati asincroni (concorrenti). Ogni flusso è indipendente dagli altri e se ci sono delle dipendenze, esse vengono gestite "sotto"

TAKE

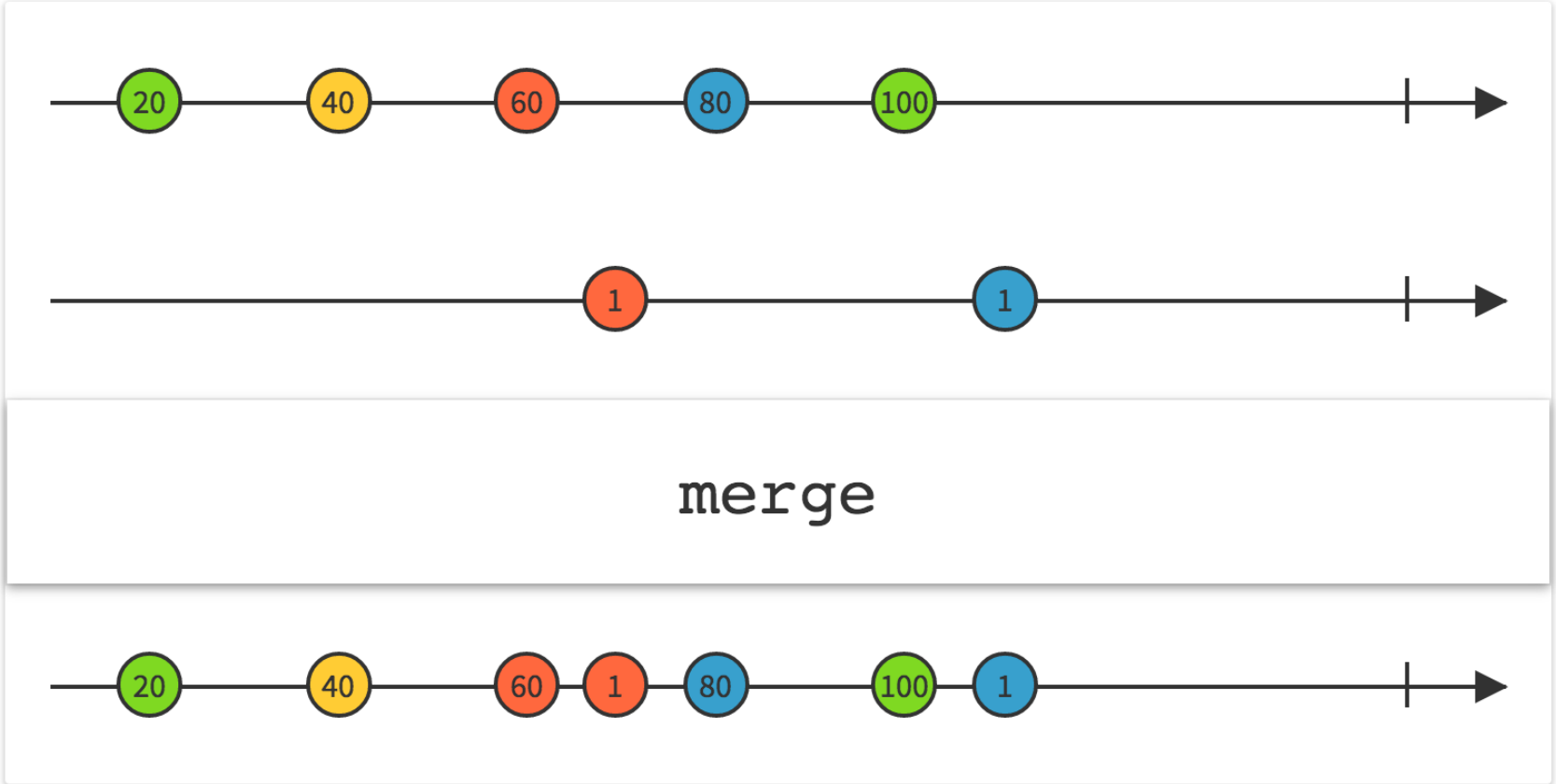


COMBINING OPERATORS

- **Merge** — combine multiple Observables into one by merging their emissions
- **Zip** — combine the emissions of multiple Observables together via a specified function and emit single items for each combination based on the results of this function
- **And/Then/When** — combine sets of items emitted by two or more Observables by means of Pattern and Plan intermediaries
- **Join** — combine items emitted by two Observables whenever an item from one Observable is emitted during a time window defined according to an item emitted by the other Observable
- **CombineLatest** — when an item is emitted by either of two Observables, combine the latest item emitted by each Observable via a specified function and emit items based on the results of this function
- **StartWith** — emit a specified sequence of items before beginning to emit the items from the source Observable
- **Switch** — convert an Observable that emits Observables into a single Observable that emits the items emitted by the most-recently-emitted of those Observables

MERGE

- Combine multiple Observables into one by merging their emissions

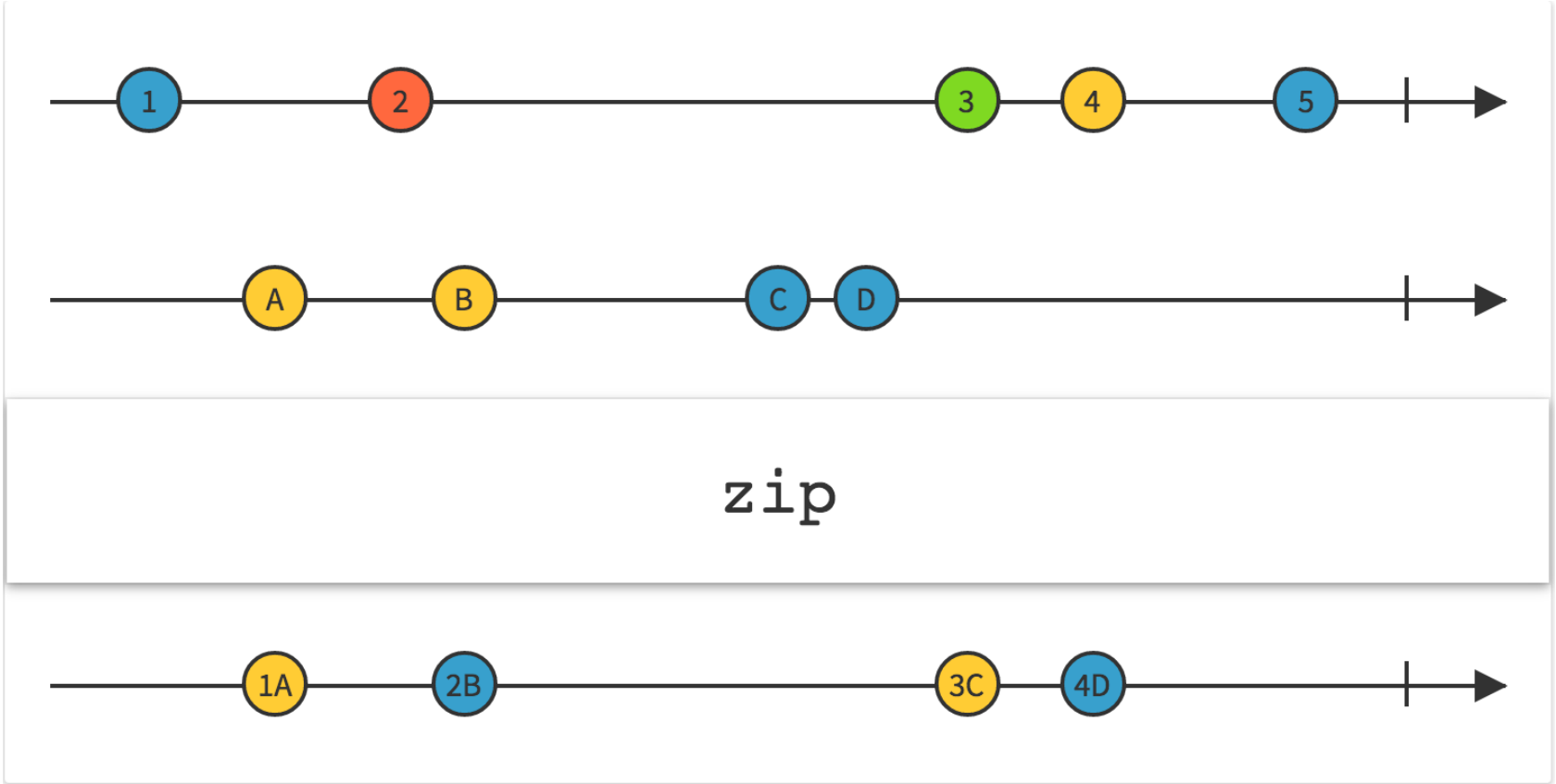


ZIP

Dati due flussi asincroni, crea un flusso asincrono dove gli elementi vengono accoppiati a due a due in modo sincrono (anche se i flussi sono asincroni)

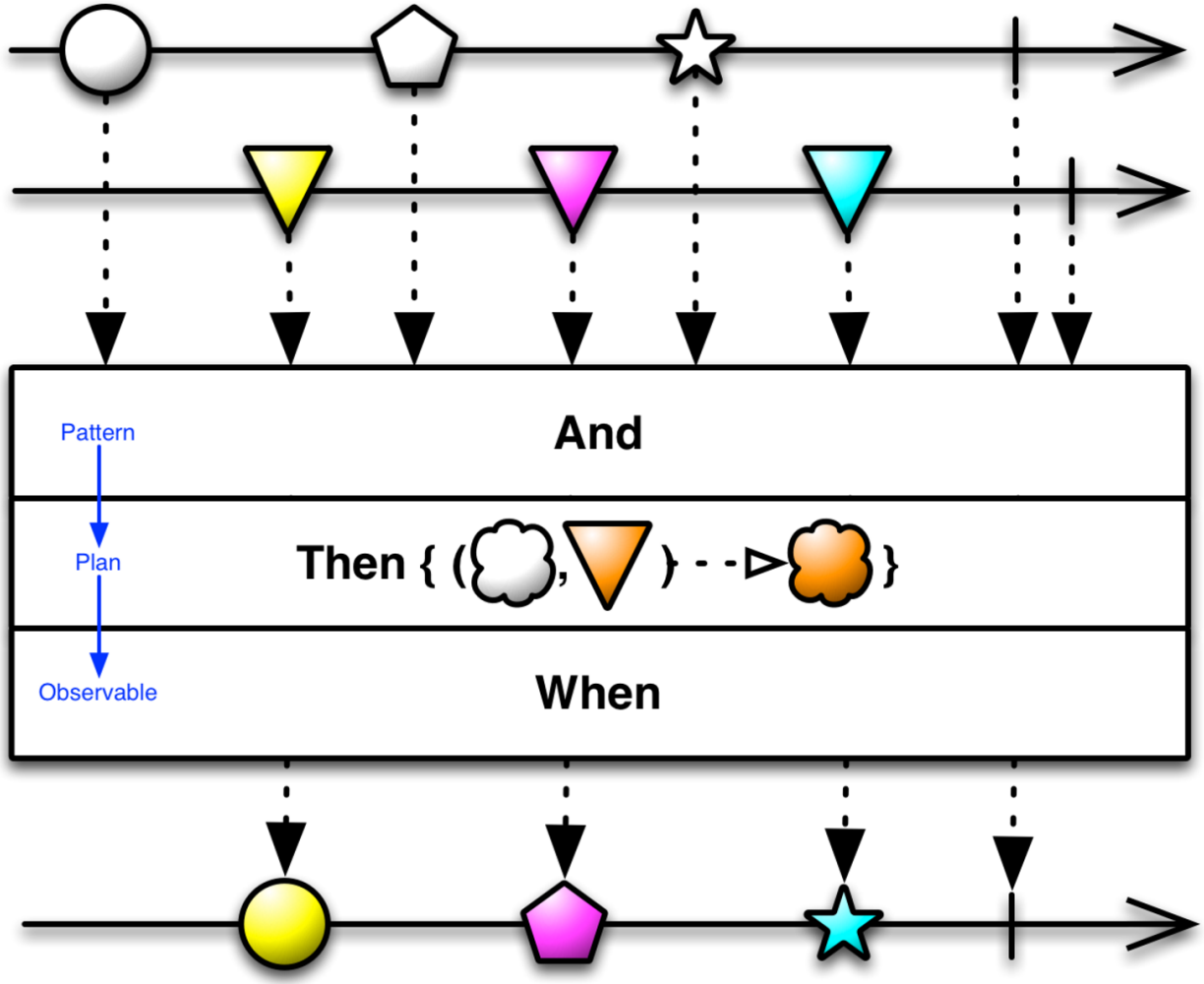
Cosa succede se un flusso è più veloce dell'altro? Zip aspetta. Se il Flusso A emette 10 elementi, ma il Flusso B ne emette solo 1, il Flusso risultante emetterà solo un singolo elemento accoppiato e terrà gli altri 9 di A "in memoria" (buffer) finché non arriva il secondo elemento di B.

- Combine the emissions of multiple Observables together via a specified function and emit single items for each combination based on the results of this function



AND/THEN/WHEN

- Combine sets of items emitted by two or more Observables by means of Pattern and Plan intermediaries



ERROR HANDLING

- **onError()** is called if an Exception is thrown at any time
- Operators don't have to handle exceptions, only subscribers do
  - so generated exceptions are propagated to subscribers

```
Observable.just("Hello, world!")
  .map(s -> potentialException(s))
  .map(s -> anotherPotentialException(s))
  .subscribe(new Subscriber<String>() {
      public void onNext(String s) { System.out.println(s); }
      public void onCompleted() { System.out.println("Completed!"); }
     public void onError(Throwable e) { System.out.println("Ouch!"); }
});
```

SUBSCRIPTIONS

- Subscription Abstraction
  - used to explicitly represent the link between an observer and a subscriber
- API to control and manage the subscription
  - **unsubscribe**
    - stop the subscription

```
Subscription subscription = Observable.just("Hello, World!")
.subscribe(s -> System.out.println(s));
   ...
subscription.unsubscribe();
```

COLD AND HOT OBSERVABLES

- **Cold** observables
  - emit signals only when it has at least one subscriber
  - pull model
    - providing items in a *lazy* way
- **Hot** observables
  - emit signals continuously, in spite of the subscribers
    - emits items at its own pace, and it is up to its observers to keep up
  - push model
    - full uncoupling between the control flow generating the items (produces) and subscribers (consumers)
  - *backpressure* techniques
    - techniques to be used when the Observer is not able to consume items as quickly as they are produced by an Observable

SCHEDULERS

- Schedulers allow to control execution on threads
- In particular, they allow to specify in which thread:
  - an Observable has to run
    - **subscribeOn**(<Scheduler>) operator
  - a Subscriber has to run
    - **observeOn**(<Scheduler>) operator

```
Flowable.fromCallable(() -> {
    Thread.sleep(1000); //  imitate expensive computation
    return "Done";
})
  .subscribeOn(Schedulers.io())
  .observeOn(Schedulers.single())
  .subscribe(System.out::println, Throwable::printStackTrace);
```

move computations or blocking IO to some other thread via subscribeOn

Once the data is ready, you can make sure they get processed on the foreground or GUI thread via observeOn

SCHEDULERS

- Available schedulers/scheduling policies
  - `Schedulers.computation()`
    - Run computation intensive work on a fixed number of dedicated threads in the background. Most asynchronous operator use this as their default Scheduler.
  - `Scheduler.io()`
    - Run I/O-like or blocking operations on a dynamically changing set of threads
  - `Schedulers.single()`
    - Run work on a single thread in a sequential and FIFO manner
- In addition, there is option to wrap an existing Executor (and its subtypes such as ExecutorService) into a Scheduler via Schedulers.from(Executor).
  - this can be used, for example, to have a larger but still fixed pool of threads (unlike computation() and io() respectively)

EXAMPLE

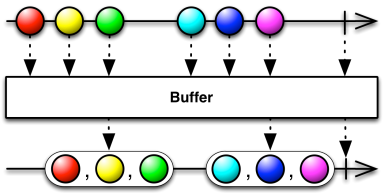
```
myObservableServices.retrieveImage(url)
  .subscribeOn(Schedulers.io())
  .observeOn(Schedulers.computation())
  .subscribe(bitmap -> processImage(bitmap));
```

BACKPRESSURE IN REACTIVEX

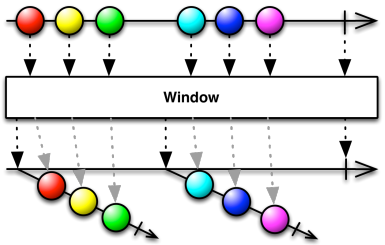
- When the dataflow runs through asynchronous steps, each step may perform different things *with different speed*
- To avoid overwhelming such steps, which usually would manifest itself as increased memory usage due to temporary buffering or the need for skipping/dropping data, a so-called *backpressure* is applied
- *Backpressure*
  - a form of ***flow control*** where the steps can express how many items are they ready to process.
  - this allows constraining the memory usage of the dataflows in situations where there is generally no way for a step to know how many items the upstream will send to it.

BACKPRESSURE STRATEGIES

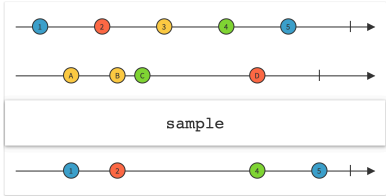
- **Buffering** overproducing observable
  - buffer(<size>) operator

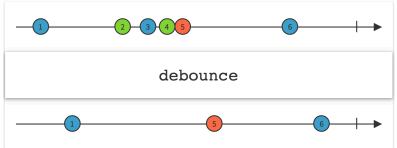


- **Batching** emitted items
  - window(<size>) operator



- **Skipping** items
  - sample(<duration>) operator
  - debounce(<duration>)
  - throttleFirst(<duration>)





HANDLING A FILLING BUFFER

- Specifying strategies to handle the case in which the buffer gets filled, generating BufferOverflowException
  - onBackpressureBuffer (<capacity>, <callback>, <strategy>)
    - strategies
      - ON_OVERFLOW_ERROR
      - ON_OVERFLOW_DROP_LATEST
      - ON_OVERFLOW_DROP_OLDEST
  - onBackpressureDrop()
    - dropping all overproduced elements

ASYNCHRONOUS PROGRAMMING: FINAL REMARKS

- Neither reactive programming or Reactive extensions can be considered the silver bullet for asynchronous programming
  - they are effective to manage asynchronous streams of data / events in a functional style…
  - ..but not as a general-purpose asynchronous programming model
- The challenge today is how to integrate all these approaches and techniques
  - synch + asynch, push + pull
  - exploiting concurrency
    - multiple communicating event loops (=> actors)
  - devising models that work also in the case of distributed systems
    - reactive distributed programming…

REFERENCES

- LEE, E. A. AND MESSERSCHMITT, D. G. 1987. Synchronous Data Flow. In Proceedings of the IEEE. Vol. 75. 1235–1245.
- ELLIOTT, C. AND HUDAK, P. 1997. Functional reactive animation. In Proceedings of the second ACM SIGPLAN International conference on Functional programming. ICFP ’97. ACM, New York, NY, USA, 263– 273.
- G. H. COOPER, S. KRISHNAMURTHI, S. 2006. Embedding dynamic dataflow in a call-by-value language. In ESOP’06: Proceedings of the 15th European conference on Programming Languages and Systems. Springer-Verlag, Berlin, Heidelberg, 294–308.
- EDWARDS, J. 2009. Coherent reaction. In OOPSLA ’09: Proceeding of the 24th ACM SIGPLAN conference companion on Object oriented programming systems languages and applications. ACM, New York, NY, USA, 925–932.
- Erik Meijer. 2012. Your mouse is a database. Commun. ACM 55, 5 (May 2012), 66-73.
- E. Bainomugisha et al. A Survey on Reactive Programming.ACM Computing Surveys (CSUR). Volume 45 Issue 4, August 2013
- G. Salvaneschi et al. Towards Distributed Reactive Programming. OOPSLA 2014
- R. Mogk et al. Fault-tolerant Distributed Reactive Programming. ECOOP 2018: 1:1-1:26